Class and Object

Class

- A class in C++ is the building block, that leads to Object-Oriented programming. It is a user-defined data type, and is a group of objects that share common properties and relationships .In C++, a class is a new data type that contains **member variables** and **member functions** that operates on the variables. A class is defined with the keyword class. It allows the data to be hidden, if necessary from external use. When we defining a class, we are creating a new abstract data type that can be treated like any other built in data type.

Class

- Generally a class specification has two parts:-
  - Class declaration
  - Class function definition
- The class declaration describes the type and scope of its members. The class function definition describes how the class functions are implemented.
- Syntax:-

```
class class-name
{
        private:
        variable declarations;
        function declaration;
        public:
        variable  declarations;
        function declaration;
};
```

Class

- The members that have been declared as private can be accessed only from with in the class. On the other hand , public members can be accessed from outside the class also. The data hiding is the key feature of oops. The use of keywords private is optional by default, the members of a class are private.
- Only the member functions can have access to the private data members and private functions

Example of class

```
class item
{
    int number;
    float cost;
    public:
    void getdata (int a ,float b);
    void putdata (void);
};
```

Object

- An Object is an instance of a Class. When a class is defined, no memory is allocated but when it is instantiated (i.e. an object is created) memory is allocated.
- Once a class has been declared we can create variables of that type by using the class name.
- Syntax: Class_name Variable_name;
- Example: item x;
  - creates a variables x of type item. In C++, the class variables are known as objects. Therefore x is called an object of type item.

Object

- Another way to create object.

```
class item
{
    int number;
    float cost;
    public:
    void getdata (int a ,float b);
    void putdata (void);
} x, y, z;
```

would create the objects x ,y ,z of type item.

Accessing Class Member

- The private data of a class can be accessed only through the member functions of that class. The main() cannot contains statements that the access number and cost directly.
- Syntax: object name.function-name(arguments);
- Example: x.getdata(100,75.5);
  - It assigns value 100 to number, and 75.5 to cost of the object x by implementing the getdata() function .
- similarly the statement x. putdata ( ); //would display the values of data members.
- x. number = 100 is illegal.
  - Although x is an object of the type item to which number belongs , the number can be accessed only through a member function and not by the object directly.

Defining Member Function

- Member can be defined in two places
  - Outside the class definition
  - Inside the class function
- **Outside the class definition**
- Member function that are declared inside a class have to be defined separately outside the class. Their definition are very much like the normal functions.
- An important difference between a member function and a normal function is that a member function incorporates a membership Identity label in the header. The '**label**' tells the compiler which class the function belongs to.

Outside the class definition

- Syntax:

```
return type class-name::function-name(argument  declaration )
{
    function-body
}
```

- The membership label class-name :: tells the compiler that the function name belongs to the class class-name . That is the scope of the function is restricted to the classname specified in the header line. The :: symbol is called scope resolution operator.

Example

```
void item :: getdata (int a , float b )
{
    number=a;
    cost=b;
}
void item :: putdata ( void)
{
    cout<<”number=:”<<number<<endl;
    cout<<”cost=”<<cost<<endl;
}
```

Inside the class function

- Another method of defining a member function is to replace the function declaration by the actual function definition inside the class .
- Difference between inside and outside defination is all the member functions defined inside the class definition are by default inline.

Example

Example:

```
class item
{
      int number;
      float cost;
      public:
      void getdata (int a ,float b)
      {
            number=a;
            cost=b;
      }
      void putdata(void)
      {
            cout<<number<<endl;
            cout<<cost<<endl;
      }
};
```

Complete Example

```
 # include< iostream. h>
Using namespace std;
class item {
        int number;
        float cost;
        public:
         void getdata ( int a , float b);
        void putdata ( void)
        {
        cout<<“Number:  ”<<number<<endl;
         cout<<”Cost : ”<<cost<<endl;
        }
};
void item :: getdata (int a , float b)
{
        number=a; cost=b;
}
int main ( )
 {
        item x;
         cout<<”\n object x:”<<endl;
         x. getdata( 100,299.95);
        x .putdata();
        item  y;
        cout<<”\n object  y”<<endl;
        y. getdata(200,175.5);
        y. putdata();
        return 0;
 }
```

Output:
object x:
Number: 100
Cost: 299.95
object y:
number : 200
Cost: 175.5

Array of Object

- An object of class represents a single record in memory, if we want more than one record of class type, we have to create an array of class or object. As we know, an array is a collection of similar type, therefore an array can be a collection of class type.
- Thus, an array of a class type is also known as an array of objects. An array of objects is declared in the same way as an array of any built-in data type.
- Syntax:
  - class_name array_name [size] ;
  - Example: item X[10];

Example

```
#include<iostream.h>
 class Employee
 {
        int Id;
        char Name[25];
        int Age;
        float Salary;
        public:
        void GetData()          //Statement 1 : Defining GetData()
        {
            cout<<"\n\tEnter Employee Id : ";
            cin>>Id;
            cout<<"\n\tEnter Employee Name : ";
            cin>>Name;
            cout<<"\n\tEnter Employee Age : ";
            cin>>Age;
            cout<<"\n\tEnter Employee Salary : ";
            cin>>Salary;
        }
        void PutData()          //Statement 2 : Defining PutData()
        {
            cout<<"\n"<<Id<<"\t"<<Name<<"\t"<<Age<<"\t"<<Salary;
        }
  };
int main()
 {
        int i;
        Employee E[3];          //Statement 3 : Creating Array of 3 Employees
        for(i=0;i<3;i++)
        {
            cout<<"\nEnter details of "<<i+1<<" Employee";
            E[i].GetData();
        }
        cout<<"\nDetails of Employees";
        for(i=0;i<3;i++)
        E[i].PutData();
         return 0;
    }
```

Output :

```
Enter details of 1 Employee
        Enter Employee Id : 101
        Enter Employee Name : Suresh
        Enter Employee Age : 29
        Enter Employee Salary : 45000

Enter details of 2 Employee
        Enter Employee Id : 102
        Enter Employee Name : Mukesh
        Enter Employee Age : 31
        Enter Employee Salary : 51000

Enter details of 3 Employee
        Enter Employee Id : 103
        Enter Employee Name : Ramesh
        Enter Employee Age : 28
        Enter Employee Salary : 47000

Details of Employees
        101     Suresh    29    45000
        102     Mukesh    31    51000
        103     Ramesh    28    47000
```

Nesting of Member Function

- We know that only the public members of a class can be accessed by the object of that class, using dot operator. However a member function can call another member function of the same class directly without using the dot operator. This is called as nesting of member functions.

Example

```
#include<iostream.h>
class set
{
        int m,n;
        public:
        void input(void);
       void display (void);
       int largest(void);
};
int set::largest (void)
{
       if(m>n)
              return m;
        else
              return n;
}
 void set::input(void)
 {
       cout<<"input values of m and n:";
       cin>>m>>n;
}
void set::display(void)
{
       cout<<"largestvalue="<<largest()<<"\n";
}
 int main()
{
       set A;
       A.input( );
       A.display( );
       return 0;
}
```

Output:
Input values of m and n: 30
17
largest value= 30

Private Member Functions

- Although it is a normal practice to place all the data items in a private section and all the functions in public, some situations may require contain functions to be hidden from the outside calls. Tasks such as deleting an account in a customer file or providing increment to employee are events of serious consequences and therefore the functions handling such tasks should have restricted access. We can place these functions in the private section.

- A private member function can only be called by another function that is a member of its class. Even an object can not invoke a private function using the dot operator.

Example

```
#include<iostream.h>
class set
{
        int m,n;
        int largest(void);
        public:
        void input(void);
        void display (void);
};
int set::largest (void)
{
        if(m>n)
                return m;
        else
                return n;
}
void set::input(void)
{
        cout<<"input values of m and n:";
        cin>>m>>n;
}
void set::display(void)
{
        cout<<"largestvalue="<<largest()<<"\n";
}
int main()
{
        set A;
        A.input( );
        A.display( );
        return 0;
}
```

Output:
Input values of m and n: 30
17
largest value= 30

Static Data Member

- We can define class members static using **static** keyword. The properties of a static member variable are similar to that of a static variable.
- Variable has contain special characteristics:-
  - It is initialized to zero when the first object of its class is created. No other initialization is permitted.
  - Only one copy of that member is created for the entire class and is shared by all the objects of that class, no matter how many objects are created.
  - It is visible only with in the class but its life time is the entire program. Static variables are normally used to maintain values common to the entire class. For example a static data member can be used as a counter that records the occurrence of all the objects.
  - int item :: count; // definition of static data member
- Note that the type and scope of each static member variable must be defined outside the class definition .This is necessary because the static data members are stored separately rather than as a part of an object.

Example

```
#include<iostream>
class   item
{
        static int count;
        int number;
        public:
                void getdata(int a)
                {
                        number=a;
                        count++;
                }
                void getcount(void)
                {
                        cout<<”count:”;
                        cout<<count<<endl;
                }
};
int item :: count ; //count defined
int main( )
{
        item a,b,c;
        cout<<"Initial Data \n";
        a.getcount();
        b.getcount();
        c.getcount();
        a.getdata(10);
        b.getdata(20);
        c.getdata(30);
        cout«"after reading data : "«endl;
        a.getcount();
        b.gelcount();
        c.getcount();
        return(0);
}
```

Output:

Initial Data:
count:0
count:0
count:0
After reading data
count: 3
count:3
count:3

Static Member Function

- By declaring a function member as static, you make it independent of any particular object of the class. A static member function can be called even if no objects of the class exist and the static functions are accessed using only the class name and the scope resolution operator ::.
- A member function that is declared static has following properties :-
  - A static function can have access to only other static members declared in the same class.
  - A static member function can be called using the class name as follows:-
  - Class_name :: function_name;

Example

```
#include<iostream>
class test
{
        int code;
        static int count; // static member variable
        public:
        void setcode()
        {
                code=++count;
        }
        void showcode()
        {
                cout<<"object member : "<<code<<end;
        }
        static void showcount()
        {
                cout<<"count="<<count<<endl;
        }
};
int test:: count;
int main()
{
        test t1,t2;
        t1.setcode();
        t2.setcode();
        test::showcount();
        test t3;
        t3.setcode();
        test::showcount();
        t1.showcode();
        t2.showcode();
        t3.showcode();
        return(0);
}
```

Output:
count = 2
Count= 3
object number:1
object number:2
object number: 3

Object as a Function Arguments

- Like any other data type, an object may be used as A function argument. This can be done in two ways
  - A copy of the entire object is passed to the function.
  - Only the address of the object is transferred to the function.
- The first method is called pass-by-value. Since a copy of the object is passed to the function, any change made to the object inside the function do not effect the object used to call the function.
- The second method is called pass-by-reference . When an address of the object is passed, the called function works directly on the actual object used in the call. This means that any changes made to the object inside the functions will reflect in the actual object .

Example

```
#include<iostream.h>
class time
{
    int hours;
    int minutes;
    public:
    void gettime(int h, int m)
    {
        hours=h;
        minutes=m;
    }
    void puttime(void)
    {
        cout<< hours<<”hours and:”;
        cout<<minutes<<”minutes:”<<end;
    }
    void sum( time ,time);
};
```

```
void time :: sum (time t1,time t2)
```

Returning Object

- A Member function can also return an object. When an object is returned from a function, a temporary object is created within the function, which holds the return value. This value is further assigned to another object in the calling function.

- The syntax for defining a function that returns an object by value is

```
class_name function_name (parameter_list)
{
    // body of the function
}
```

Example

```
#include<iostream>
using namespace std;
class weight {
  int kilogram;
  int gram;
  public:
    void getdata ();
    void putdata ();
    weight sum_weight (weight) ;
} ;
void weight :: getdata() {
  cout<<“\nKilograms:";
  cin>>kilogram;
  cout<<"Grams:";
  cin>>gram;
}
 void weight :: putdata () {
   cout<<kilogram<<" Kgs. and  "<<gram<<" gros.\n";
 }
 weight weight :: sum_weight(weight w) {
  weight temp;
  temp.gram = gram + w.gram;
  temp.kilogram=temp.gram/1000;
  temp.gram=temp.gram%1000;
  temp.kilogram+=kilogram+w.kilogram;
  return(temp);
}
int main () {
```

Friend Function

- C++ supports the feature of encapsulation in which the data is bundled together with the functions operating on it to form a single unit. By doing this C++ ensures that data is accessible only by the functions operating on it and not to anyone outside the class.
- But in some real-time applications, sometimes we might want to access data outside.
- C++ provides a facility for accessing private and protected data by means of a special feature called **friend** function or class.
- A friend function in C++ is a function that is preceded by the keyword **friend**. When the function is declared as a friend, then it can access the private and protected data members of the class.

Friend Function

- The friend function is declared inside the class whose private and protected data members are to be accessed. The function can be defined anywhere in the code file and we need not use the keyword friend or the scope resolution operator.
- Syntax:

```
class className
{
        ……
      friend returnType  functionName(arg list);
};
```

Rules for friend function

- A friend function can be declared in the private or public section of the class.
- It can be called like a normal function without using the object.
- A friend function is not invoked using the class object as it is not in the scope of the class.
- A friend function cannot access the private and protected data members of the class directly. It needs to make use of a class object and then access the members using the dot operator.
- Usually object as parameter of a friend function.
- A friend function can be a global function or a member of another class.

Example

```
#include<iostream.h>
class sample
{
int a;
int b;
public:
void setvalue(int x,int y)
{
        a=x;
        b=y;
}
void display()
{
        cout<<"A="<<a<<endl;
        cout<<"B="<<b<<endl;
}
friend float mean( sample s);
};

float mean (sample s)
{
```

Friend function in different classes

```
#include<iostream.h>
class abc;
class xyz
{
        int x;
        public:
void setvalue(int i)
{
        x= i;
}
void display()
{
        cout<<"X:"<<x<<endl;
}
friend void max (xyz, abc);
};
class abc
{
        int a;
        public:
void setvalue( int i)
{
        a=i;
}


void display()
```

Friend class

- Just like friend functions, we can also have a friend class. Friend class can access private and protected members of the class to which it is a friend.
- Syntax:

```
class A{
……
friend class B;
};
class B{
……..
};
```

- As depicted above, class B is a friend of class A. So class B can access the private and protected members of class A.

- But this does not mean that class A can access private and protected members of the class B.

Example

```
#include <iostream>
using namespace std;
class Area
{
  int length,breadth,area;

  public:

  void getArea(int l,int b)
  {
      length=l;
      breadth=b;
  }
  void calcArea()
  {
    area = length * breadth;
  }

  friend class printClass;
};
class printClass
```